Heaps and Treaps

James Anderson (updated by Cemil Kirbas)

Data Structures

CS 3100/5100

Wright State University

Some updates are based on the presentation by Mr. Abdul Bari **on YouTube**

Motivation

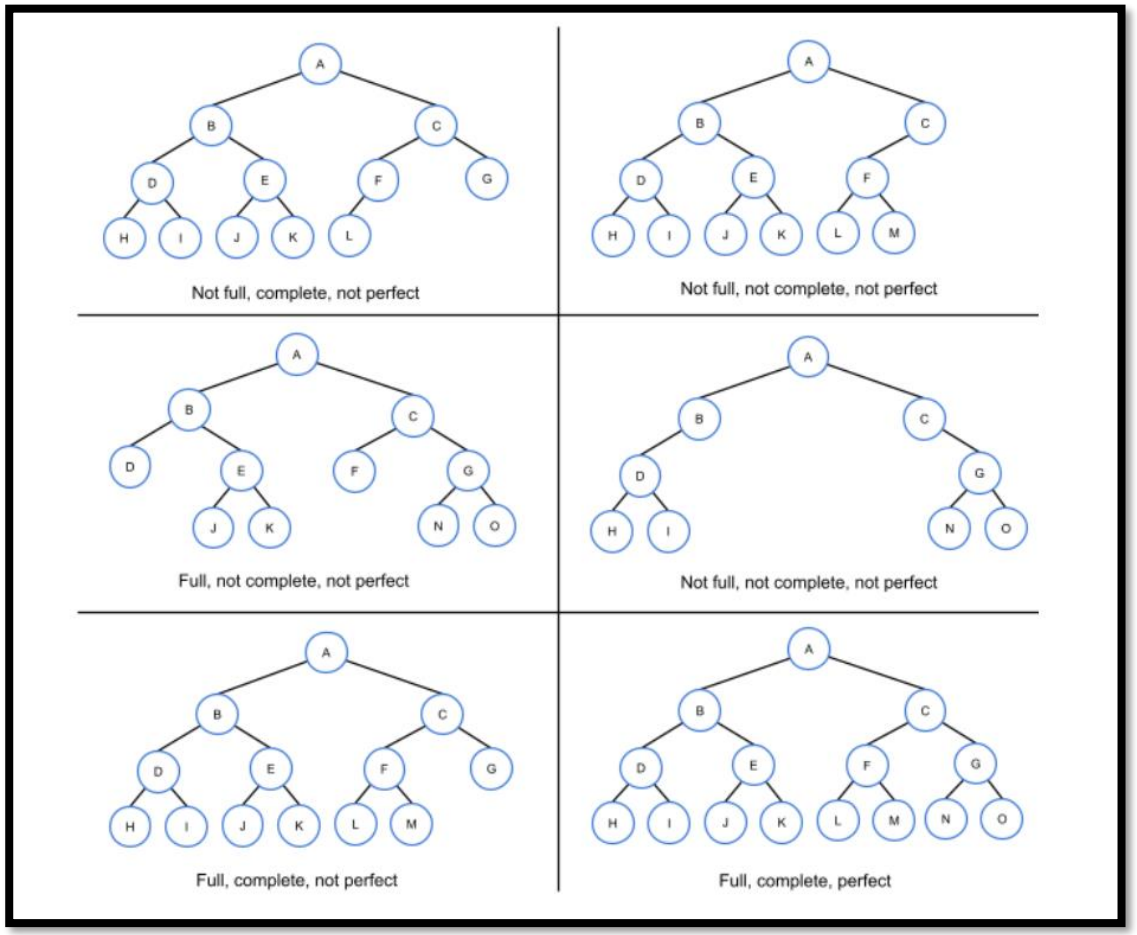
Some data structures can be specialized to do one and only one thing really well

Items change often, always want fast access to the current min (or max)

Priority queue – jobs added with varying priorities, always want the one with the highest priority

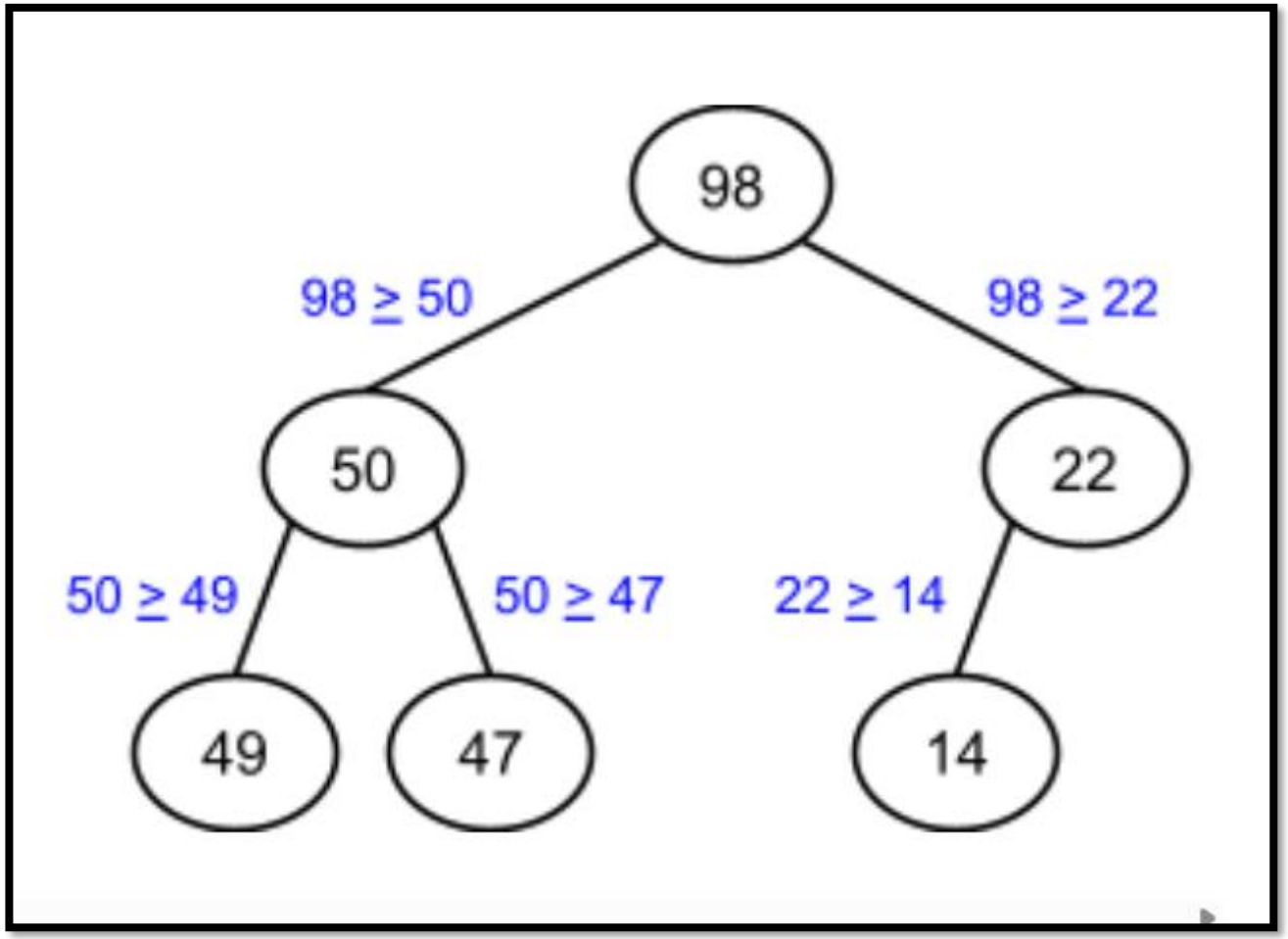
Special Types of Binary Trees

- **Full**: every node contains 0 or 2 children
- **Complete**: all levels (except possibly the last) contain all possible nodes, nodes in last level are as far left as possible
- **Perfect**: All internal nodes have 2 children, leaves are at same level



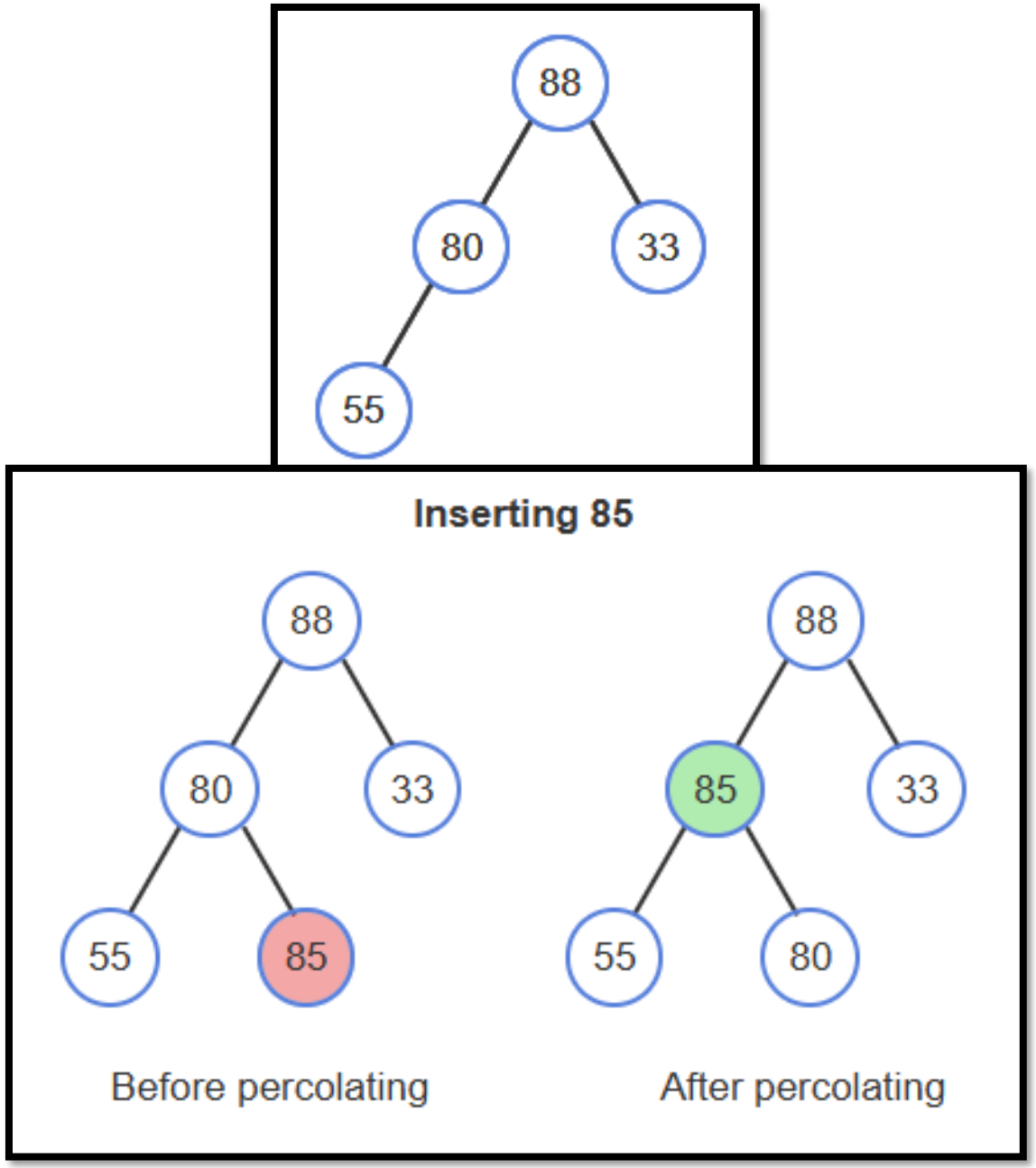
Heap Concept

- Complete binary tree
- Max-heap
  - All node keys are ≥ to children's keys
- Min-heap
  - All node keys are ≤ to children's keys



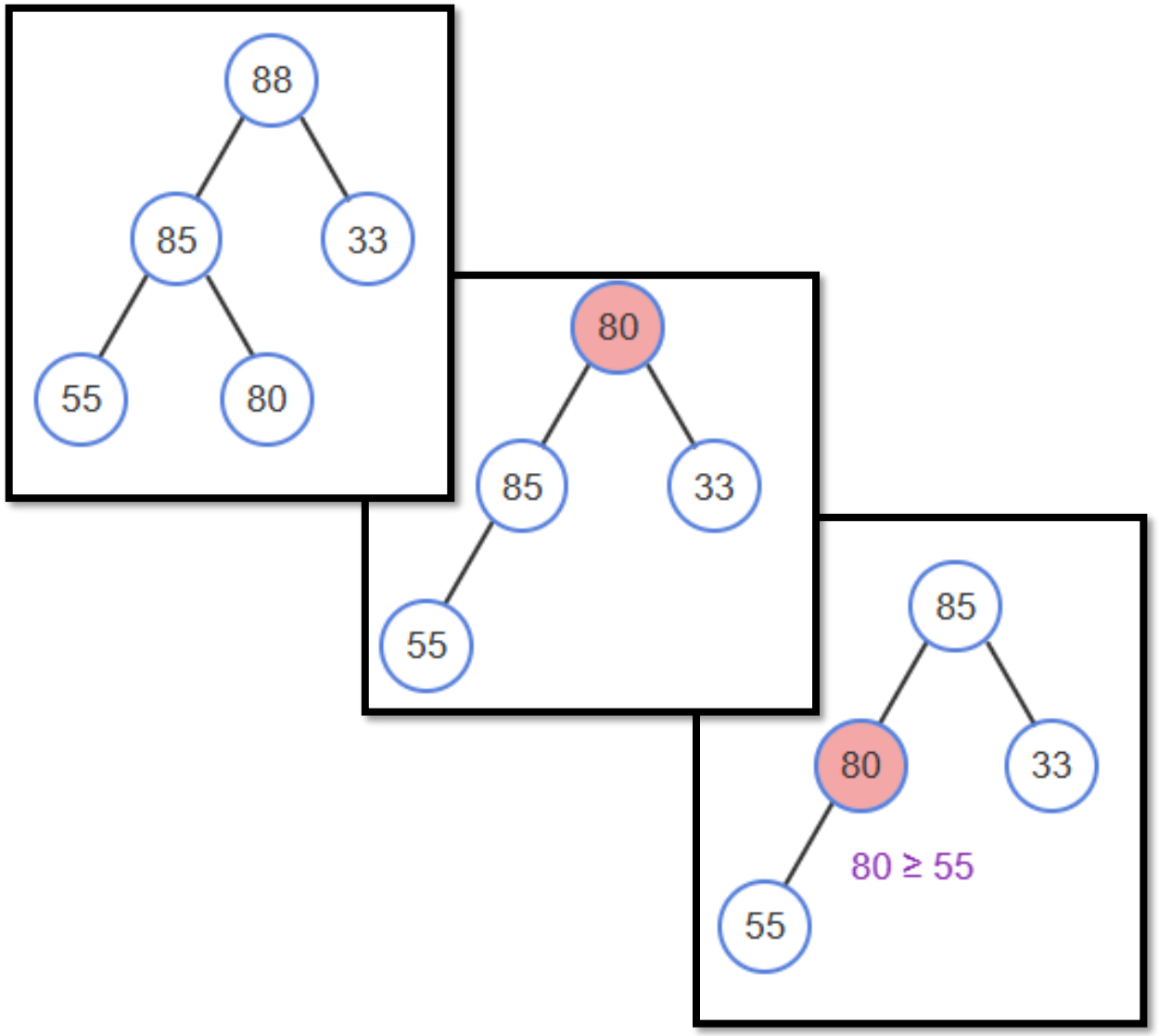
Max-Heap Insert

- Because heap is complete binary tree, any new node must go at the last level, as far left as possible
- Insert new value in that position, swap with parent until parent is larger
- Percolate-up
- Min-Heap is same, except percolate-up until parent smaller



Max-Heap Remove

- Always remove the largest – root node
- Only node we can remove is last level, farthest right
- Put value of that node into root, swap down until larger than both children
- If larger than both, swap with the largest
- Min-heap same, swap until smaller

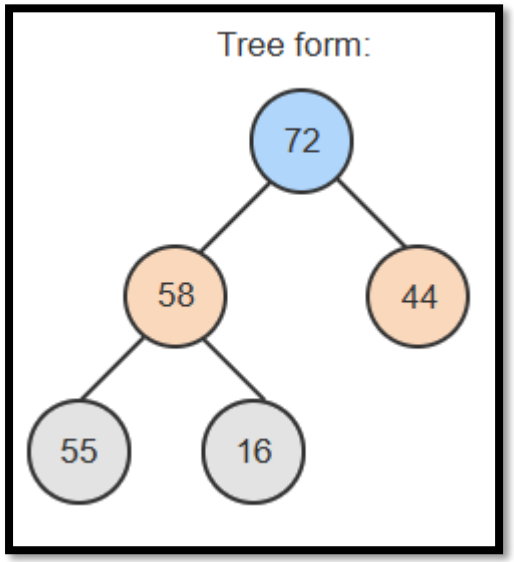


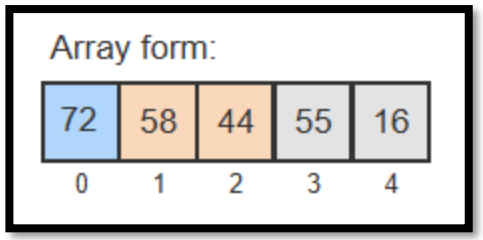
Insert and Remove Time Complexity

- Complete binary tree – minimal levels
- Worst case - $O(\log_2 n)$
- If we just want the largest (or smallest) value, we can just look at the root - $O(1)$
- Actual remove can occur on a separate thread, while main thread continues after peeking at largest
- Search?

Heaps Using Arrays

- Complete binary tree – values inserted top-bottom, left-right
- Each node value at a corresponding index
  - Root: 0
  - Root->left: 1
  - Root->right:2
- New value always goes at next index in array, value at last index placed into index 0 when removing





Parent and Child Indices

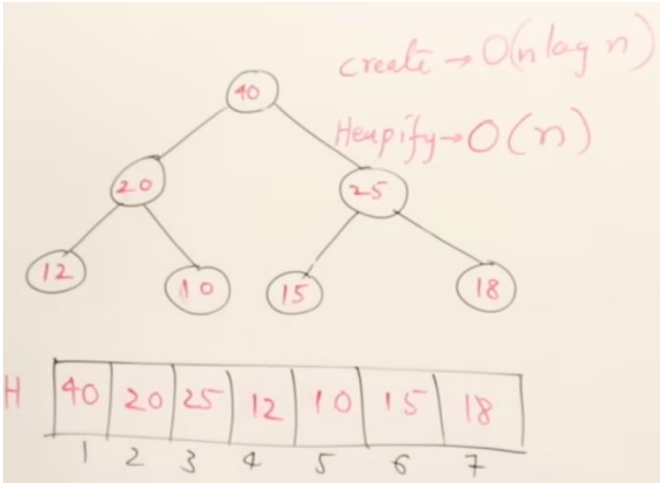
Node Index	Parent Index	Left Child Index	Right Child Index
0	N/A	1	2
1	0	3	4
2	0	5	6
3	1	7	8
4	1	9	10
5	2	11	12
…	…	…	…
i	$\left\lfloor \frac{i-1}{2} \right\rfloor$	$2i+1$	$2(i+1)$

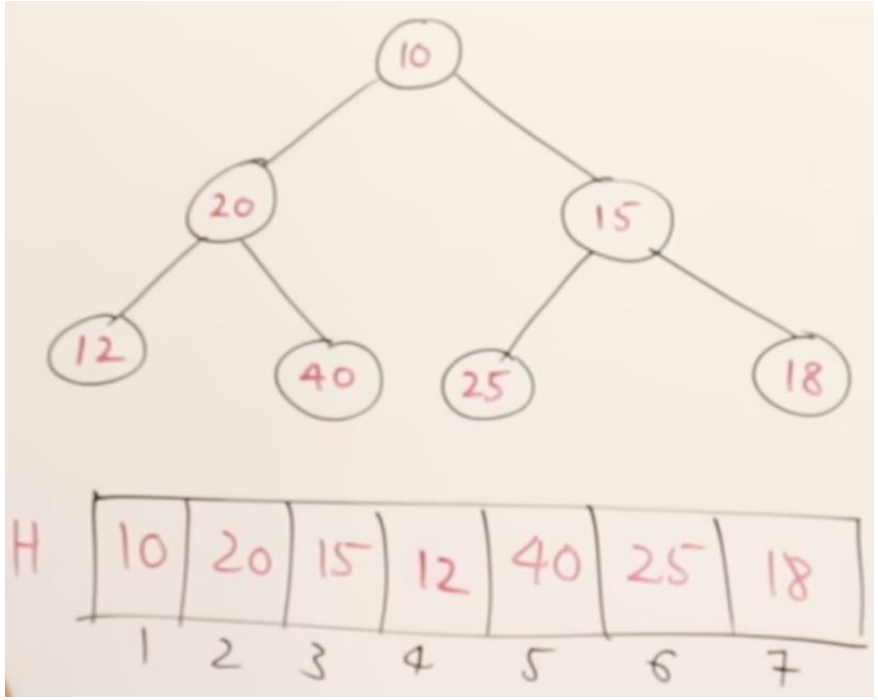
Heapify (aka BuildHeap)

- To create a max-heap from an existing array –
  - Insert all values into an empty heap - $O(n \log_2 n)$
- Alternative
  - Leaf nodes already satisfy heap property
  - Start with first non-leaf internal node
  - Percolate-down: swap with largest child that is greater than value
  - Repeat with nodes *right to left, bottom to top*
- Heapify in array: largest internal node index: $\left\lfloor \frac{n}{2} \right\rfloor - 1$

Heapify

- Process of creating a heap given a set of keys using bottom-up processing.





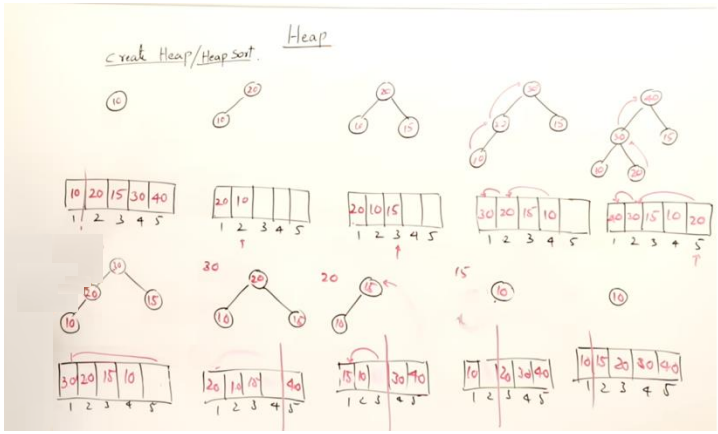
Heapsort

- Given array of unordered elements – first heapify
- Repeatedly remove from heap to obtain elements largest to smallest
- Sort in-place: keep unsorted part of array (the heap) and the sorted part
- Swap root with last element in unsorted part, that becomes sorted

Heap Sort

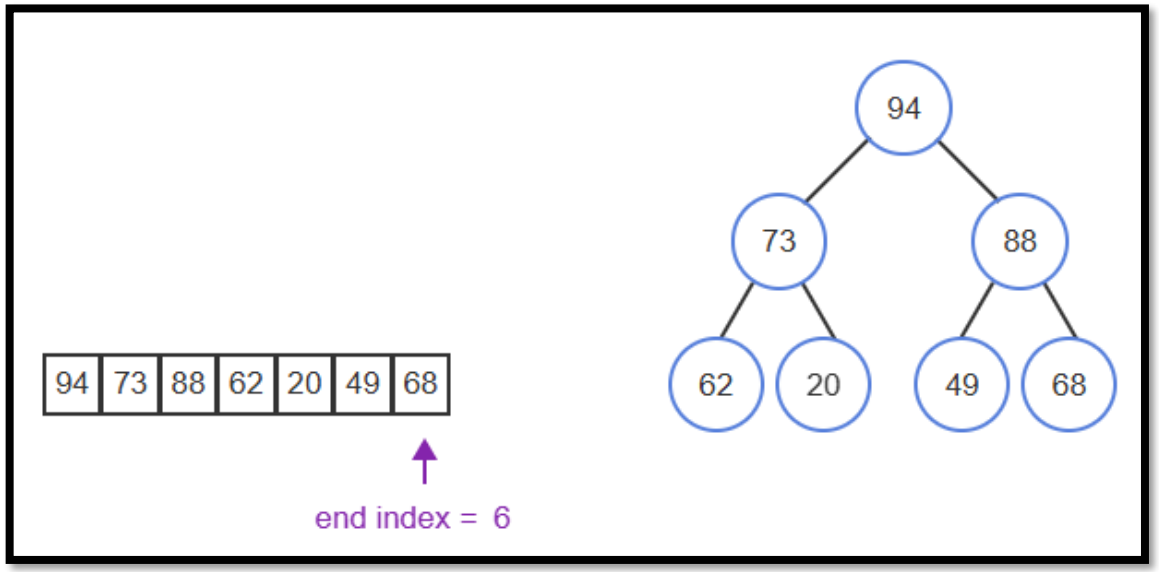
Steps:

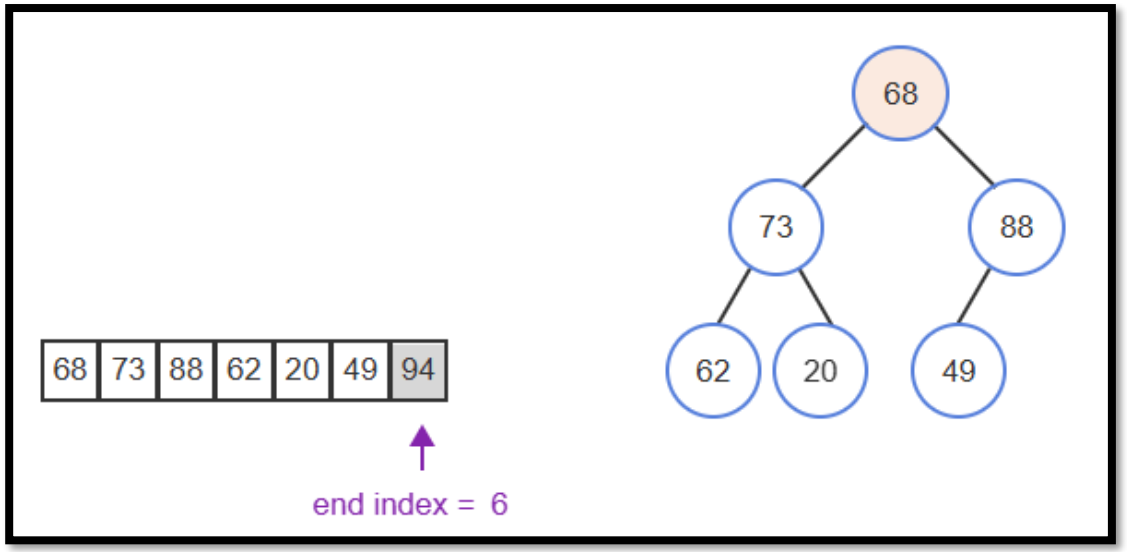
- ❑ Create a heap given a set of keys.
- ❑ Delete the top node (root) one-by-one, and add the deleted node to the emptied spot in the array.



Heapsort

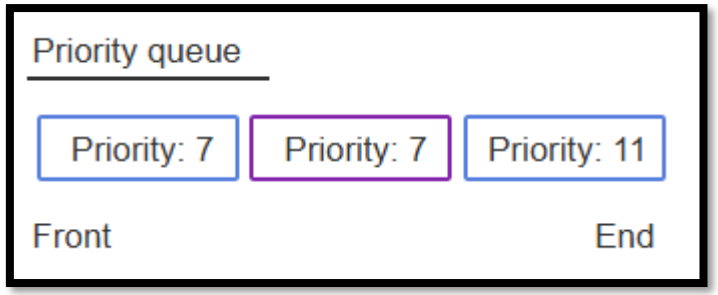
- End index tracks end of heap/beginning of sorted

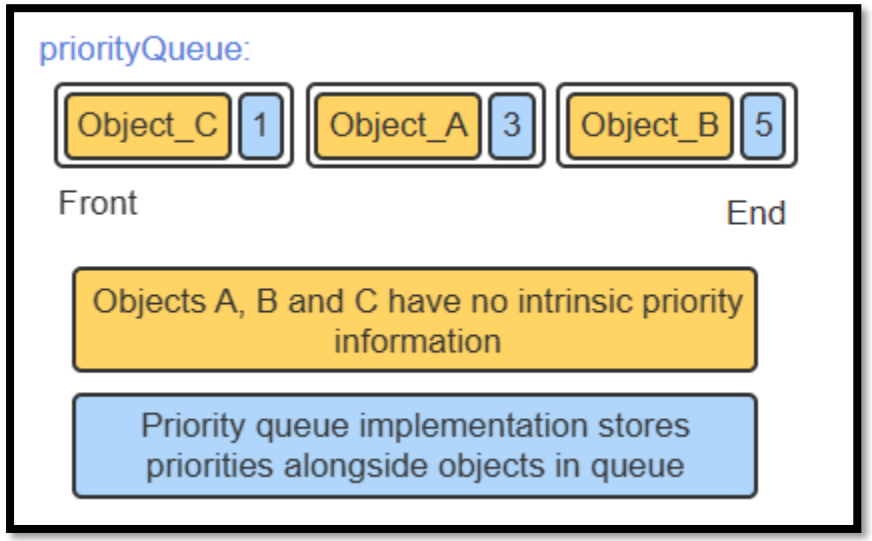




Priority Queue ADT

- **Enqueue**: insert into queue after all other lower priority
- **Dequeue**: remove element with highest priority
- **Peek**: Look at but don't remove item at front
- Priority can be part of element, or can enqueue-with-priority





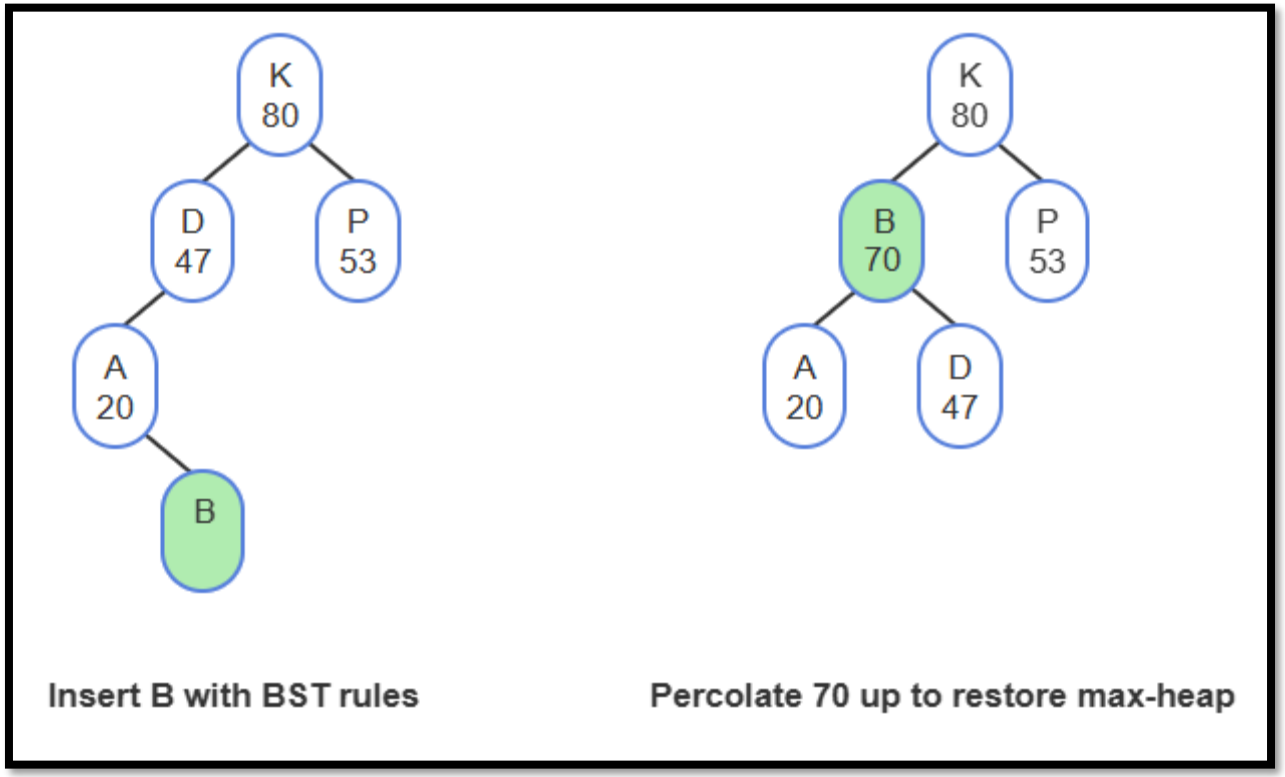
Implementing Priority Queue with Heap

Priority Queue Operation	Heap functionality used to implement operation	Worst-case runtime complexity
Enqueue	Insert	$O(\log_2 n)$
Dequeue	Remove	$O(\log_2 n)$
Peek	Return value in root	$O(1)$
Is-empty	Return true if no nodes in heap, false otherwise	$O(1)$
Get-length	Return number of nodes (or heap's length)	$O(1)$

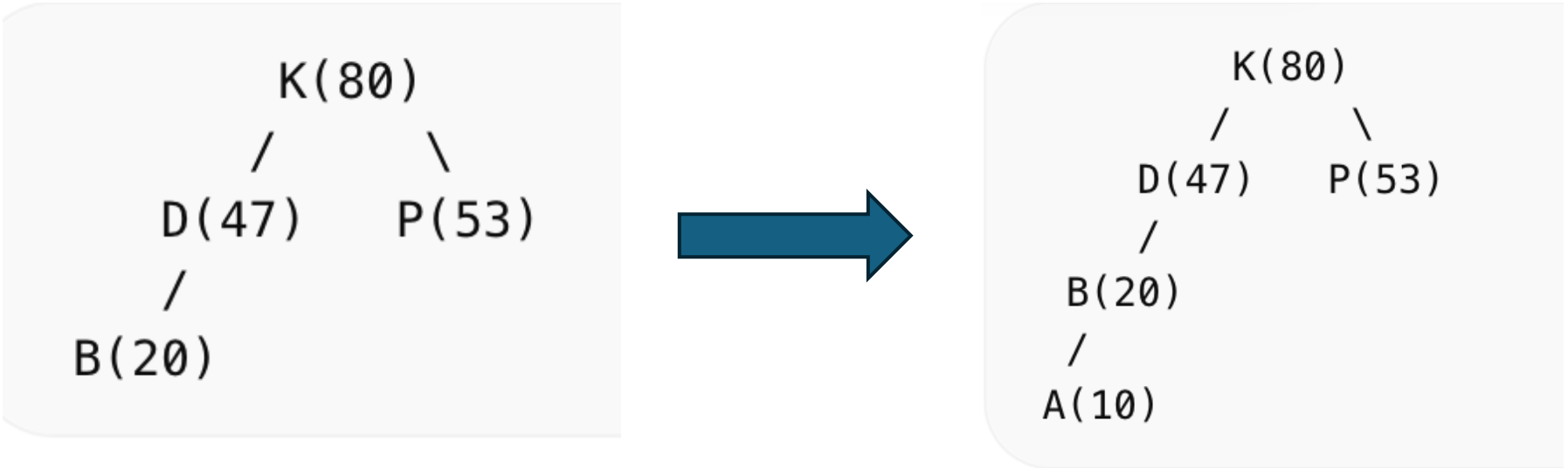
Treaps

- BST problem: inserts can cause out-of-balance
- One solution: randomize order of insertion
- We can’t randomize the key, but we could create a random value
- Treap: BST on *keys* + heap on randomly generated “*priorities*”
- Search: same as BST using key
- Insert: same as BST using key, then a random priority used to maintain a heap by rotating node up (using AVL rotation)
- Remove: set node priority to $-\infty$, rotate node down until leaf

Treap Insert



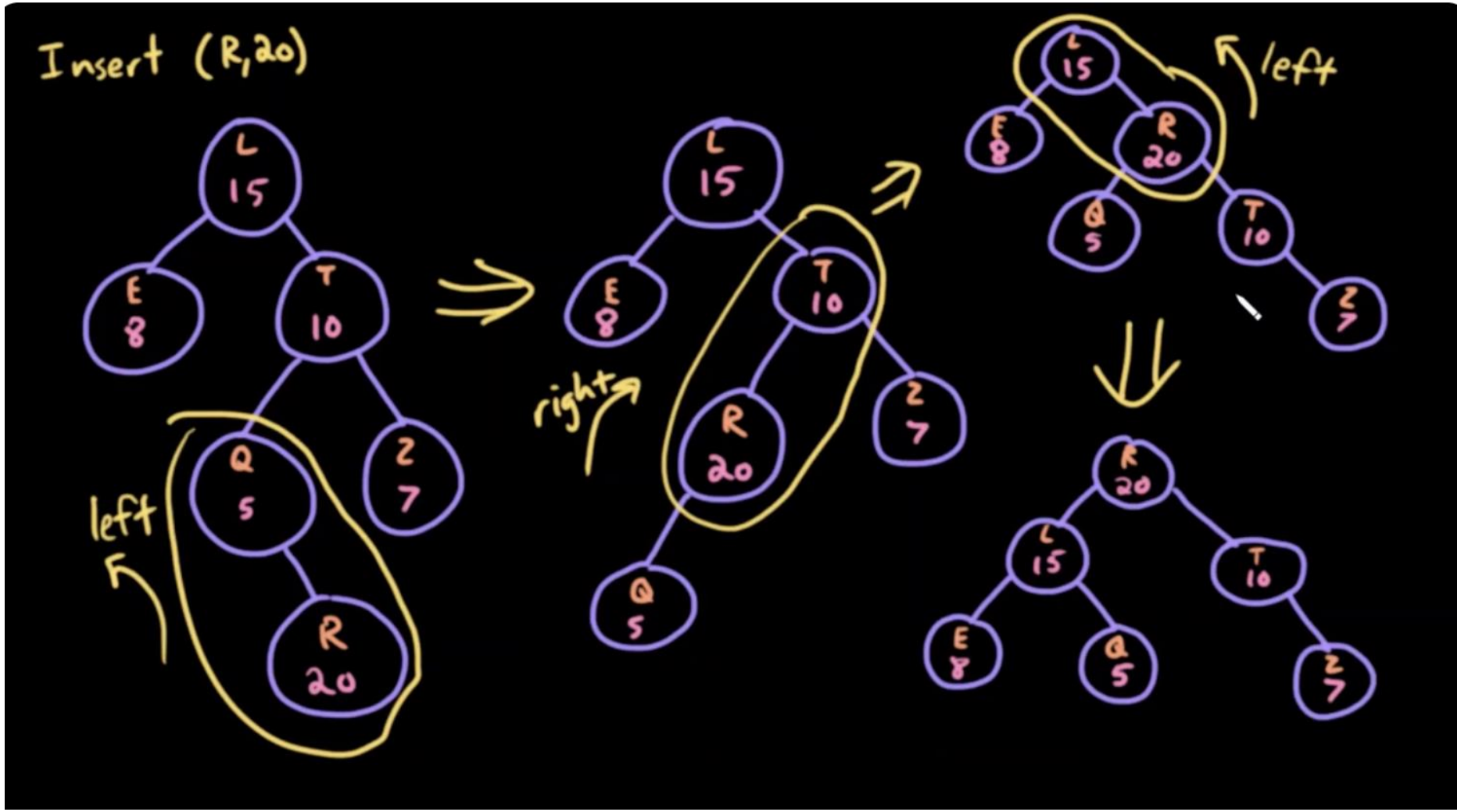
Treap Insert: Insert node A with priority 10



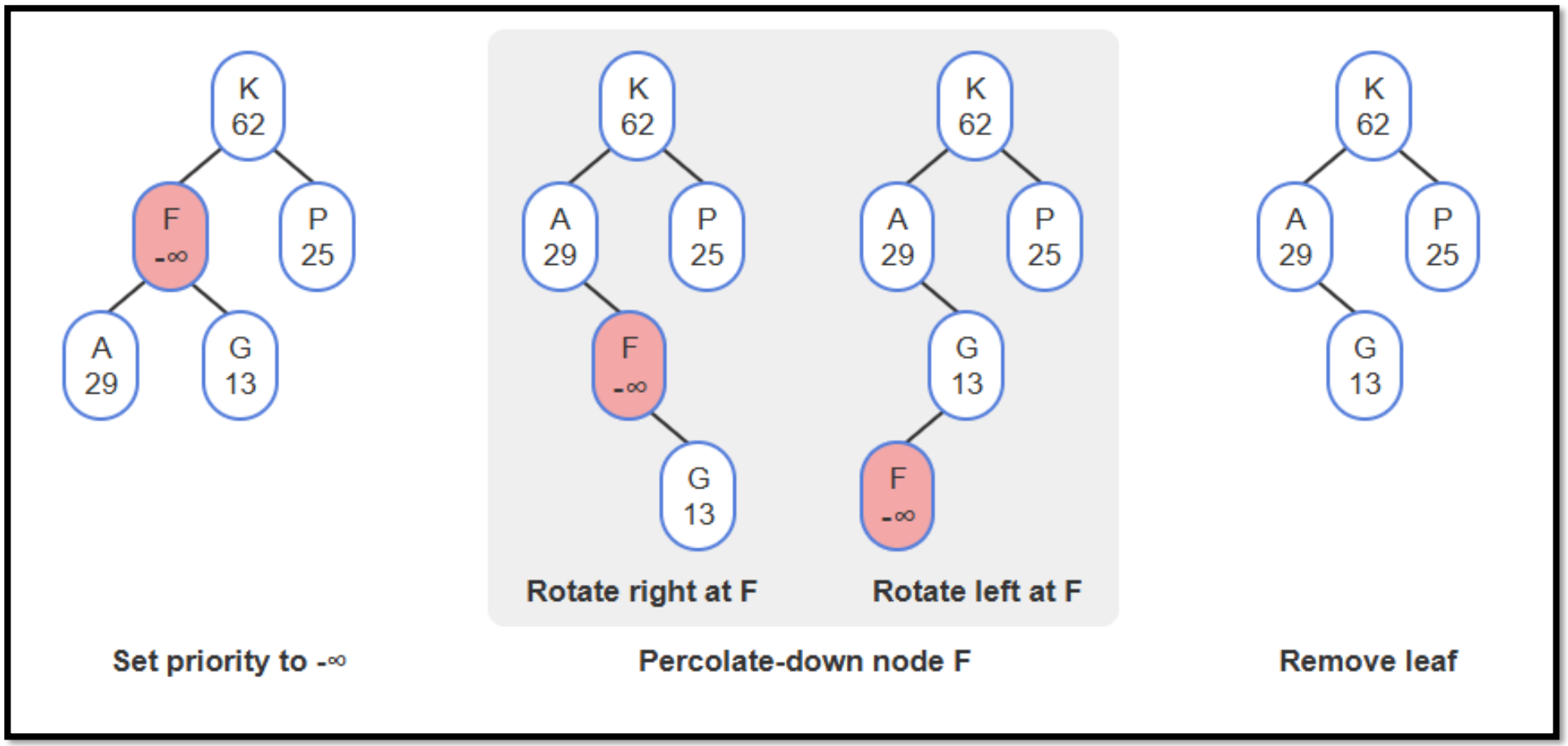
What treaps actually guarantee:

- Treaps maintain:
  - **BST property (by key)**
  - **Heap property (by priority)**
- That's it. No "must be balanced" rule written anywhere.

Treap Insert



Treap Remove



C++ priority_queue

- priority_queue<int>

Member function	Usage and description	Example starting with examplePQ: 89 51 26 Front End
push()	`pqInstance.push(item)` Enqueues an item into the priority queue.	`examplePQ.push(42);` examplePQ: 89 51 42 26 Front End
pop()	`pqInstance.pop()` Removes the priority queue's highest priority item. Does not return anything.	`examplePQ.pop(); // removes 89` examplePQ: 51 26 Front End

top()	`pqInstance.top()`
	
	Returns the queue's highest priority item. Does not remove the item.
		`cout << examplePQ.top();`
		
		Output:
		`89`

size()	`pqInstance.size()`
	
	Returns the number of items in the priority queue.
		`cout << examplePQ.size();`
		
		Output:
		`3`